

IEEE CS Bangalore Chapter Internship and Mentorship Program - 2026

Duration: 1st April 2026 to 30th September 2026

Monthly Progress Report - 1

Project ID : P107

Project Title: ASL-ISL Translation Engine (AITE)

Name Of Students:

- Naman Nagar
- Monisha Sharma
- Nandita R Nadig

University/ College Name: PES University

Mentor Name: Dr. Sudhamani M J

Mentor's Organization: PES University

Progress of the Project

1. Problem Definition
2. Literature Review
3. Existing System
4. Proposed System
5. Knowledge gained - Tools, Technology, Courses etc.
6. Architectural Framework
7. Project Implementation
8. Results
9. Conclusion and Future Work
10. Research article preparation (if applicable)

Github repository: <https://github.com/charliee03/ASL-ISL>

1. Problem Definition

Deaf individuals rely on sign languages such as ASL (American Sign Language) and ISL (Indian Sign Language) for communication. However, these languages differ significantly in syntax, gestures, and grammar, making cross-lingual communication extremely difficult. Unlike spoken languages, sign languages lack standardized translation systems, especially for real-time applications.

Gap Identified: No existing system performs cross-sign-language translation (ASL to ISL) that handles syntactic and gestural differences in real-time with video inputs.

Goal: Build a 3-stage modular pipeline — Sign Recognition (ASL to gloss), Cross-Lingual Translation (ASL gloss to ISL gloss), and Sign Generation (ISL gloss to avatar animation).

2. Literature Review

Completed a comprehensive review of **30 research papers (2018–2026)** covering sign language recognition, translation, and generation. Key findings:

- **Closest to our pipeline:** Camgoz et al. (2020) - Sign Language Transformers (joint end-to-end recognition and translation); Kumar et al. (2024) - Enhanced ASL to ISL Translation

- **Recognition baselines:** Li et al. (2020) - WLASL dataset and word-level recognition; De Coster et al. (2020) - Transformer-based recognition; Bohacek & Hruz (2022) - Sign pose transformer
- **Generation references:** Stoll et al. (2021) - NMT + GAN for sign production; Saunders et al. (2020) - Progressive transformers; Kissel et al. (2021) - Pose-guided GAN
- **Key gap confirmed:** No real-time cross-sign-language translation system handling ASL to ISL exists

Full literature review table saved in docs/literature_review.md.

3. Existing System

Current sign language systems primarily:

- Focus on sign-to-text or text-to-sign within a single language
- Use datasets like WLASL but lack cross-sign mapping
- Do not support real-time bidirectional translation
- Ignore grammar differences between sign languages

Examples studied:

System	Limitation
Neural Sign Language Translation (Camgoz, 2018)	Limited dataset scale; not cross-lingual
Text2Sign (Stoll, 2021)	Avatar realism limitations
SignNet II (Chaudhary, 2023)	Dataset/domain limitations
Enhanced ASL-ISL (Kumar, 2024)	Prototype-level, not production-ready

4. Proposed System

AITE Pipeline (3 stages):

1. **Sign Recognition** - Transformer-based model trained on WLASL, extracts ASL gloss from video via MediaPipe hand keypoints

- 2. **Cross-Lingual Translation** - Lightweight quantized LLM transforms ASL grammar to ISL
- 3. **Sign Generation** - GAN-based avatar renders ISL in real-time (>=15 FPS)

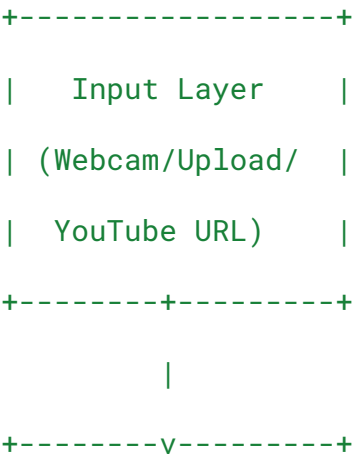
Input sources: Webcam, video upload, YouTube URL

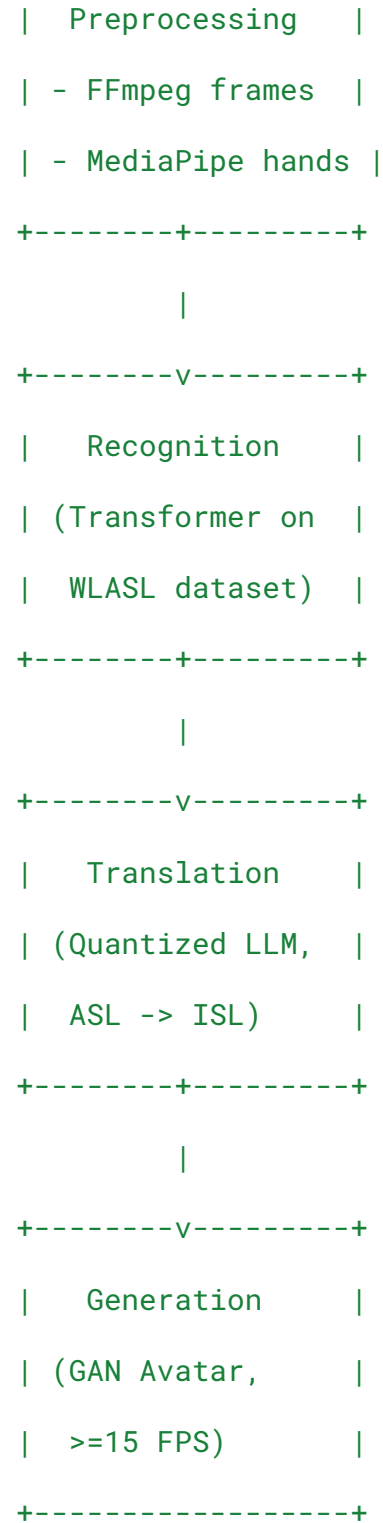
Output: Avatar animation displaying ISL signs

5. Knowledge Gained — Tools, Technologies, Courses

Area	Tools/Libraries Learned
Hand Tracking	MediaPipe Hands (21 keypoint detection, real-time landmark extraction)
Deep Learning	PyTorch (dataset loaders, Transformer architecture)
Computer Vision	OpenCV (frame extraction, video processing, image I/O)
Web APIs	FastAPI (REST endpoint design for keypoint extraction)
Video Processing	FFmpeg (frame sampling, video encoding/decoding)
Project Structure	Modular Python project design, config management with YAML

6. Architectural Framework





The Architectural Framework for the ASL-ISL Translation Engine (AITE) operates as a clear, sequential pipeline designed to convert American Sign Language (ASL) video input into Indian Sign Language (ISL) avatar animation.

4. **Input and Preprocessing:** The journey begins with the Input Layer, accepting video streams from a webcam, uploaded file, or YouTube URL. The video is processed using FFmpeg to extract frames, and MediaPipe Hands is used to perform high-speed hand tracking, extracting 21 keypoints (x, y, z coordinates) from each hand. This process converts the raw visual data into structured, numerical features.
 5. **Recognition:** The feature data is fed into the Recognition module, where a Transformer-based deep learning model, trained on the WLASL dataset, interprets the ASL gestures and converts them into an intermediary text format known as ASL gloss.
 6. **Cross-Lingual Translation:** The ASL gloss moves to the Translation module. Here, a lightweight quantized Large Language Model (LLM) is used to analyze the linguistic differences and perform grammar transformation, accurately converting the ASL gloss into ISL gloss.
 7. **Sign Generation:** The final Generation module uses a GAN (Generative Adversarial Network) driven avatar to render the translated ISL gloss into a fluid, real-time animation, targeting a frame rate of 15 FPS or greater for effective visual communication.
-

7. Project Implementation

Completed (Weeks 1–4):

Repository Setup:

- Initialized Git repository with modular project structure
- Created Python package layout (src/recognition, src/translation, src/generation, src/utils, src/api)
- Configured dependencies (requirements.txt, pyproject.toml)
- Added YAML configs for each pipeline stage

MediaPipe Integration (Week 3–4):

- Implemented HandKeypointExtractor in src/recognition/preprocess.py — extracts 21 hand landmarks (x, y, z) per hand from video frames
- Created HandTrackingDemo in src/utils/mediapipe_demo.py — real-time webcam demo with landmark visualization (press 'q' to quit, 's' to save frame)
- Added batch video processing support

Feature Extraction:

- `src/utils/feature_extractor.py` — batch processing of video datasets to extract and save MediaPipe keypoints
- Supports uniform and stride-based frame sampling strategies
- Saves extracted features as `.npy` files with metadata JSON

Dataset Loaders:

- `src/recognition/dataset.py` — PyTorch Dataset classes for WLASL and ISL datasets
- Uniform frame sampling, automatic keypoint extraction, train/test split support

API Server:

- `src/api/server.py` — FastAPI server with `/health`, `/extract-keypoints`, and `/predict-gloss` endpoints
- Real-time keypoint extraction from uploaded frames

Exploration Notebooks:

- `notebooks/01_mediapipe_exploration.ipynb` — MediaPipe Hands exploration
- `notebooks/02_dataset_exploration.ipynb` — WLASL/ISL dataset structure analysis

Testing:

- `tests/test_preprocess.py` — Unit tests for `HandKeypointExtractor`

Directory Structure:

ASL-ISL/

```

├─ src/
|   └─ recognition/
|       └─ model.py           # Transformer architecture
|       └─ preprocess.py      # Hand keypoint extraction
|       └─ dataset.py         # WLASL/ISL dataset loaders
|   └─ translation/
|       └─ translator.py      # LLM translation wrapper

```

```

|   |— generation/
|   |   |— avatar.py           # GAN generator/discriminator
|   |— api/
|   |   |— server.py          # FastAPI server
|   |— utils/
|       |— video.py           # FFmpeg frame extraction
|       |— metrics.py         # BLEU, WER evaluation
|       |— mediapipe_demo.py  # Real-time demo script
|       |— feature_extractor.py # Batch feature extraction
|— configs/                   # YAML pipeline configs
|— docs/                      # Literature review, reports
|— notebooks/                 # Jupyter notebooks
|— scripts/                   # Dataset setup scripts
|— tests/                     # Unit tests
|— pyproject.toml
|— requirements.txt

```

Key Code Snippets:

Hand Keypoint Extraction (src/recognition/preprocess.py):

```

class HandKeypointExtractor:

    def extract(self, frame):

        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        results = self.hands.process(rgb)

        keypoints = []

        if results.multi_hand_landmarks:

```



```

        for hand in results.multi_hand_landmarks:
            for lm in hand.landmark:
                keypoints.extend([lm.x, lm.y, lm.z])

    return keypoints

Dataset Loader (src/recognition/dataset.py):

class WLASLDataset(Dataset):
    def _sample_frames(self, video_path):
        cap = cv2.VideoCapture(str(video_path))
        total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        indices = np.linspace(0, total - 1, self.num_frames,
dtype=int)

        keypoints = []

        for idx in indices:
            cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
            ret, frame = cap.read()

            if not ret: continue

            kp = self.extractor.extract(frame)

            keypoints.append(kp)

        return np.stack(keypoints)

```

8. Results

Component	Status	Details
Repository setup	Done	Modular Python project with configs, tests, notebooks

Component	Status	Details
Literature review	Done	30 papers analyzed, saved in docs/literature_review.md
Dataset identification	Done	WLASL and ISL-CSLGR identified; setup scripts ready
MediaPipe integration	Done	Hand tracking with 21 keypoints, real-time demo working
Feature extraction	Needs refinement	Batch processing pipeline for video to keypoints
Dataset loaders	Done	PyTorch Dataset classes for both datasets
API server	In progress	FastAPI with keypoint extraction endpoints
Unit tests	In progress	Basic preprocessing tests for handpoint extraction

9. Conclusion and Future Work

Conclusion (Weeks 1–4): Successfully completed project setup, literature review, MediaPipe integration, and feature extraction pipeline. The modular architecture is in place for all three pipeline stages.

Future Work (Weeks 5–8):

- Train Transformer model for ASL recognition on WLASL
- Evaluate recognition accuracy (WER metric)
- Develop ASL-to-ISL translation module using quantized LLM
- Implement grammar transformation rules

10. Research Article Preparation

Not yet started. Planned for Weeks 9+.

Signatures**Approved By:**

Name	Signature	Date
Dr. Sudhamani M J		